

GD Assistant — Project Plan

Build a Windows desktop AI assistant called **GD Assistant**.

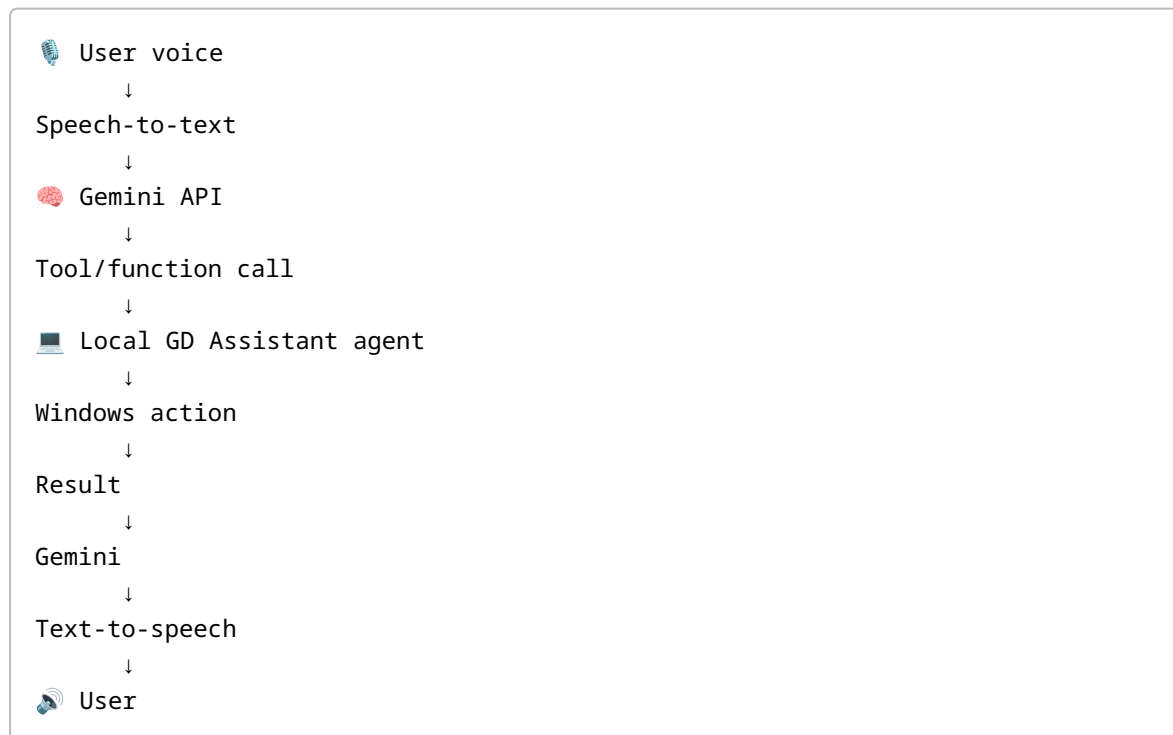
Goal

Create a Siri-like personal assistant for Windows that:

- Accepts voice input.
- Uses **Google Gemini API as the cloud LLM/brain**.
- Performs actions **locally on the user's PC**.
- Can eventually understand screenshots/screen content.
- Can eventually control the mouse and keyboard.
- Has persistent local memory.
- Can eventually be packaged as a standalone `.exe` so friends can use it.

The user's PC is not powerful enough for practical local LLM inference, so **do not require a local LLM**. Gemini handles reasoning; the local application handles execution.

Core Architecture



Gemini should NEVER have unrestricted access to the PC.

Instead, expose a controlled list of local tools/functions. Gemini can request a tool, but the local application decides whether the tool is permitted and executes it.

Planned Technology

Initial implementation:

- **Python**
- Gemini API
- Gemini function/tool calling
- Local Windows APIs / Python libraries for actions
- Local speech-to-text
- Local text-to-speech if practical
- JSON configuration/memory
- PyInstaller later for `.exe` packaging

Keep dependencies reasonably lightweight.

Initial Tool System

Start with safe tools such as:

- `open_app(name)`
- `close_app(name)`
- `get_system_stats()`
- `search_files(query)`
- `open_url(url)`
- `get_current_time()`

Later add:

- `take_screenshot()`
- `get_processes()`
- `get_disk_usage()`
- `set_volume()`
- `type_text()`
- `press_key()`
- mouse control
- arbitrary commands, but ONLY behind strong confirmation/security controls

Security

This is extremely important.

NEVER execute arbitrary Python, PowerShell, CMD, or shell commands just because Gemini requested them.

Use an explicit tool whitelist.

Potentially dangerous actions must require user confirmation, especially:

- deleting files
- modifying system settings

- shutting down/restarting
- executing arbitrary commands
- installing/uninstalling software
- interacting with sensitive files

Do not hard-code the developer's Gemini API key into the application.

The eventual distributed `.exe` should allow each user to enter their own Gemini API key, so the developer's quota/key is not exposed.

Store user configuration locally, preferably under something like:

```
%APPDATA%\GD Assistant\
```

Possible files:

```
config.json
memory.json
permissions.json
```

Never log or expose API keys.

Development Phases

Phase 1 — Basic Gemini client

Create a minimal Python program that: 1. Accepts text input. 2. Sends it to Gemini. 3. Prints the response. 4. Handles API errors and rate limits gracefully.

Phase 2 — Function calling

Add the local tool system.

Example:

User:

Open Notepad.

Gemini:

Calls `open_app("Notepad")`.

Local agent:

Validates the request and opens Notepad.

Return the result to Gemini.

Phase 3 — Voice

Add:

- microphone input
- speech-to-text
- Gemini conversation
- text-to-speech

The goal is a natural voice assistant.

Phase 4 — Memory

Add controlled persistent memory.

Examples:

- user preferences
- frequently used applications
- custom commands
- user-defined locations/paths

Memory should remain local unless the user explicitly chooses otherwise.

Phase 5 — Screen awareness

Allow the assistant to capture a screenshot and send it to Gemini when requested.

Example:

"What's wrong with this error?"

The assistant captures the screen, Gemini analyzes it, and returns an explanation.

Do NOT continuously upload screenshots by default.

Phase 6 — Computer control

Add controlled mouse/keyboard tools.

Examples:

- click
- type
- press keys
- scroll

Require confirmation for potentially dangerous interactions.

Phase 7 — Packaging

Package the application using PyInstaller into:

```
GD-Assistant.exe
```

The resulting application should not require Python to be installed.

Provide a first-run setup screen for:

- Gemini API key
- microphone permissions
- optional assistant name
- tool permissions

Important Design Principles

1. **Gemini = brain**
2. **Local Python application = hands**
3. **User = final authority**
4. **No unrestricted computer access**
5. **Privacy by default**
6. **Local memory**
7. **No hard-coded API keys**
8. Build incrementally and test every phase before adding the next.

First Milestone

Do NOT build the entire assistant at once.

Start with a clean, small project that can:

```
User types message
      ↓
Gemini API
      ↓
Gemini response
      ↓
Terminal output
```

Then add exactly **one safe local function**, preferably `open_app()`.

Keep the code understandable and explain important implementation decisions so the project can also serve as a programming-learning project.